

Sesión 2: Circuitos Booleanos

~~Def~~ Los circuitos booleanos son modelos computacionales que computan funciones booleanas.

~~Def~~ Def (Circuitos): Sea Φ un conjunto de funciones booleanas. Un circuito ~~sea~~ de n variables sobre la base Φ es una secuencia $g_1, \dots, g_t$ de $t \geq n$ funciones booleanas tales $g_1 = x_1, \dots, g_n = x_n$ y $\forall i \geq n$, g_i es una aplicación $g_i = \varphi(g_{i_1}, \dots, g_{i_d})$ donde $\varphi \in \Phi$ y $i_1, \dots, i_d < i$.

Un circuito en general se representa de forma "bottom-up".

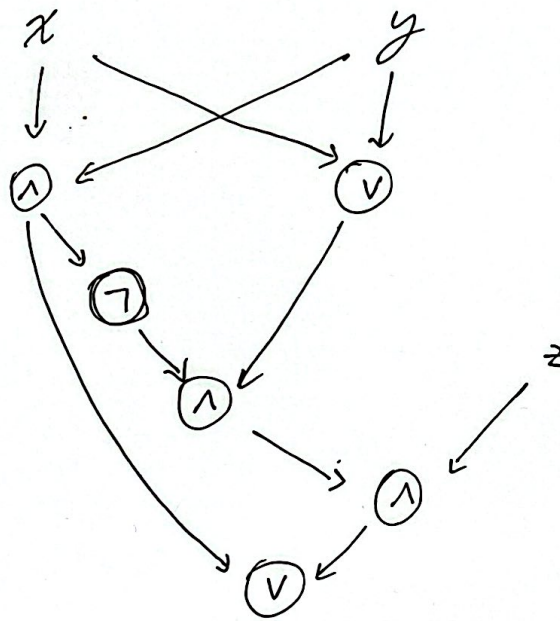
El tamaño (size) de un circuito es el ~~total~~ número total $t - n$ de "gates" ^{o nodos} (no contamos los variables inputs).

~~Def~~ La profundidad (depth) de un circuito es el ^{longo del} camino más largo entre una variable input y el nodo output.

Def (Computar): Decimos que un circuito computa una función booleana f (o output) si esta está entre los g_i .

Cada circuito puede ser visto como un DAG con función-0 nodos (grado 0) que corresponden a las variables, y cada otro nodo v corresponde a una función $\varphi \in \Phi$. Uno o más nodos son outputs.

Ejemplo 1: Un circuito sobre $B = \{\wedge, \vee, \neg\}$ computando $\text{Maj}_3(x, y, z) = 1$ iff $x + y + z \geq 2$.



Si $zz = 6$
Depth = 5
~~2~~

Def (Formula): Una fórmula es un circuito en cual todos los nodos tienen formato (constantes de salida) a lo más 1.

Obs: El grafo de una fórmula es un arbol. La dif crucial entre una fórmula y un circuito es que en el circuito un resultado puede ser usado múltiples veces sin necesidad de recomputarlo.

Def (DeMorgan Circuit): Un circuito DeMorgan es un circuito sobre la base $\{1, \vee\}$ pero los inputs pueden ser la variable y sus negaciones.

Obs: Usando las leyes de DeMorgan $\neg(X \vee Y) = \neg X \wedge \neg Y$ y $\neg(X \wedge Y) = \neg X \vee \neg Y$, se puede mostrar que cualquier circuito sobre $\{1, \vee, \neg\}$ puede ser reducido a la forma DeMorgan, a lo mas doblando el numero de nodos ~~ya que~~ (la prof se montiene).

Def:

Def (Circuitos probabilísticos): [Un circuito se dice probabilístico si además de tener los variables inputs estándar $x_1, \dots, x_n$, tiene algunos inputs $r_1, \dots, r_m$ llamados inputs aleatorios.] Cuando estos random inputs son elegidos de una distribución uniforme $\{0,1\}$, el output $C(x)$ del circuito es una variable 0-1 aleatoria.

~~Un circuito probabilístico~~

Def (Computar una función): Un circuito probabilístico $C(x)$ computa una función booleana $f(x)$ si: $\text{Prob}[C(x) = f(x)] \geq 3/4, \forall x \in \{0,1\}^n$.

Obs: Preferimos usar $3/4$ o cualquier prob $> 1/2$.

Pregunta: Pueden los circuitos probabilísticos computar una función usando ~~una~~ usando un tamaño mucho menor que un circuito determinístico?

Lema: (Majority trick): Si $x_1, \dots, x_m$ son R.V Bernoulli independientes con probabilidad de éxito $1/2 + \epsilon$, entonces ~~Prob~~

$$\text{Prob}[\text{Maj}(x_1, \dots, x_m) = 0] \leq e^{-2\epsilon^2 m}$$

Pf Sea F la colección de subconjuntos de $[m]$ ~~de~~ de tamaño $> m/2$.

$$\begin{aligned} \Rightarrow \text{Prob}[\text{Maj}(x_1, \dots, x_m) = 0] &= \sum_{S \in F} \text{Prob}[x_i = 0, \forall i \in S] \cdot \text{Prob}[x_i = 1, \forall i \notin S] \\ &= \sum_{S \in F} (1/2 - \epsilon)^{|S|} (1/2 + \epsilon)^{m-|S|} \leq \sum_{\substack{|S| > m/2 \\ (1/2 - \epsilon) < (1/2 + \epsilon)}} (1/2 - \epsilon)^{m/2} (1/2 + \epsilon)^{m/2} = \sum_{S \in F} \underbrace{\left(1/4 - \epsilon^2\right)^{m/2}}_{(1/4 - \epsilon^2)^{m/2}} \end{aligned}$$

(4)

$$\leq 2^m \cdot (1/4 - \epsilon^2)^{m/2} = 2^{m/2} \cdot 2^{m/2} \cdot (1/4 - \epsilon^2)^{m/2} = (1 - 4\epsilon^2)^{m/2} \leq e^{-2\epsilon^2 m}$$

2^m es
el total
de subconjuntos
de $[m]$

Teorema (Adleman 1978): Si una función booleana f de n variables puede ser computada por un circuito probabilístico de tamaño M , entonces f puede ser computada por un circuito determinístico de tamaño a lo más $8 \cdot n \cdot M$.

Proof 1. Sea C un circuito prob que computa f , entonces

$$\text{Prob}[C(x) = f(x)] \geq 3/4, \quad \forall x \in \{0,1\}^n.$$

Tomamos m copias independientes de C : $C_1, \dots, C_m$.

Consideramos al circuito probabilístico C' que computa la función majority de los outputs de los m circuitos $C_1, \dots, C_m$.

Si fijamos $a \in \{0,1\}^n$ y definimos x_i como la variable aleatoria indicatriz " $C_i(a) = f(a)$ "

$$\left. \begin{aligned} x_1 &:= \mathbb{1}_{C_1(a) = f(a)} \\ &\vdots \\ x_m &:= \mathbb{1}_{C_m(a) = f(a)} \end{aligned} \right\}$$

Para cada variable aleatoria tenemos que $\text{Prob}[x_i = 1] = 1/2 + \epsilon$

$$\text{Prob}[x_i = 1] = 1/2 + \epsilon$$

Por lema (Majority trick): $\text{Prob}[\text{Maj}(x_1, \dots, x_m) = 0] \leq e^{-2\epsilon^2 m}$. (5)

En otros palabras, el circuito C' ~~se equivoca~~ se equivoca en el input $a \in \{0,1\}^n$ con probabilidad a lo mas $e^{-2\epsilon^2 m} = e^{-m/8}$.

~~Por Union Bound~~ $\text{Prob}[C' \text{ se equivoca } a] \leq e^{-m/8}$.

Ahora analizamos ~~se~~ ~~es~~ cual es la probabilidad que C' se equivoca con algún input en $\{0,1\}^n$.

$$\text{Prob}[C' \text{ se equivoca con algún input en } \{0,1\}^n] \\ = \text{Prob}[B_{a_1} \text{ or } B_{a_2} \text{ or } B_{a_3} \dots B_{a_{2^n}}] \quad \text{con } B_{a_i} := \{C' \text{ se equivoca con } a_i\}$$

$$\text{Prob}[B_{a_1} \text{ or } \dots \text{ or } B_{a_{2^n}}] \underset{\text{U-B}}{\leq} \sum_{a \in \{0,1\}^n} e^{-m/8} = 2^n \cdot e^{-m/8}.$$

Tomando $m = 8n$ vemos que $2^n \cdot e^{-m/8} < 1$.
 $\Rightarrow \text{Prob}[C' \text{ da la respuesta correcta } \forall w \in \{0,1\}^n] > 0.$

$\Rightarrow$ Existe un set de ~~inputs aleatorios~~ inputs aleatorios que logran que C' proporcione respuesta correcta en todos los inputs $\{0,1\}^n$. (Este circuito es determinístico).

Average time of computation

Sea $C = (g_1, \dots, g_s)$ un circuito computando una función booleana $f(x)$, es decir, $g_s(x) = f(x)$. Podemos considerar la noción de "tiempo de computación" en un input $a \in \{0,1\}^n$.

Para esto, introducimos una variable booleana z (llamada la variable output), que es un resultado parcial en el circuito $z = g_i(a)$. Nuestro objetivo es interrumpir la secuencia de cómputo $g_1(a), \dots, g_s(a)$ cuando la variable output ya tiene el valor correcto $z = f(a)$.

Para esto, ~~introducimos~~ declaramos algunos nodos como "nodos de parada". Dado $a \in \{0,1\}^n$ una computación $g_1(a), \dots, g_s(a)$ continúa hasta que el primer nodo g_i es un nodo de parada y $g_i(a) = 1$. Entonces la computación a a termina y el output $C(a)$ del circuito es el valor actual de z .

Def (Tiempo de computación): El tiempo de computación $t_c(a)$ de un circuito en el input a es el número i de nodos evaluados hasta que el valor es computado. El tiempo promedio de un circuito C es
$$t(C) = 2^{-n} \sum_{a \in \{0,1\}^n} t_c(a).$$

Ejemplo:

$$Z = 1$$

$$f_1 = x_1 \text{ (stop)}$$

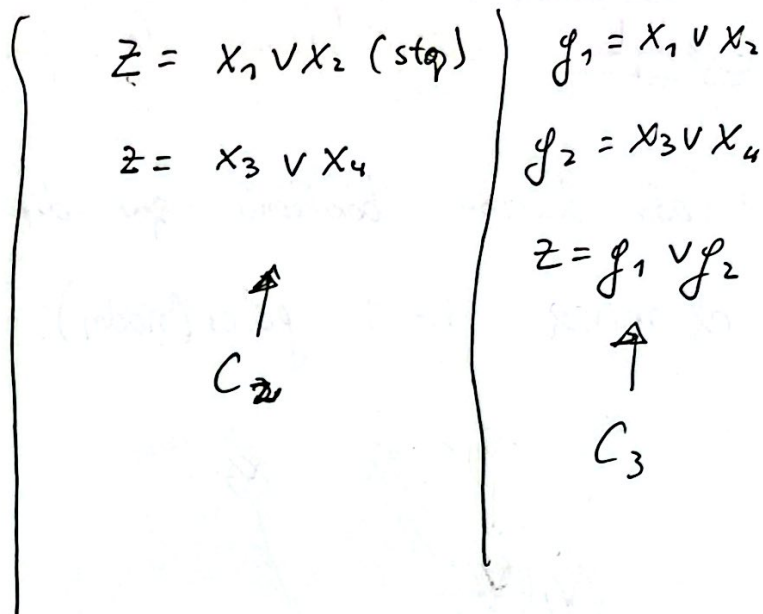
$$f_2 = x_2 \text{ (stop)}$$

$$f_3 = x_3 \text{ (stop)}$$

$$f_4 = x_4 \text{ (stop)}$$

$$Z = 0$$

$C_1 \rightarrow$



3 circuitos que computan OR en $(x_1 \vee x_2 \vee x_3 \vee x_4)$. Sobre

$a = (0, 1, 0, 0)$; el circuito 1 toma $t_c(a) = 3$, el segundo $t_c(a) = 1$

y el tercero $t_c(a) = 3$. El tiempo promedio del circuito 3 es

$t(C_3) = 3$, en cambio, para el circuito 2 $t(C_2) = \frac{1}{16}(2 + 4 \cdot 2)$

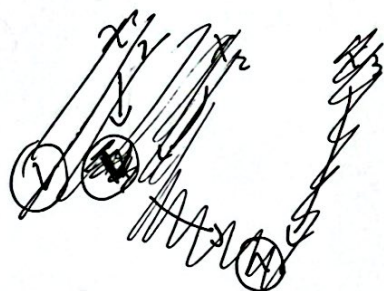
Def (tiempo promedio de f): El tiempo promedio de una función booleana $t(f)$ es el tiempo promedio mínimo de un circuito computando f .

Obs: Siempre se tiene que $t(f) \leq C(f)$.

Chazhkin (1997) proved that boolean functions f of n variables requiring $t(f) = \Omega(2^n/n)$ exists.

Ejemplo: Consideremos la función $Th_2^n(x)$.

- Cada función booleana que depende en sus n variables requiere al menos $n-1$ gates (nodos). $\Rightarrow C(Th_2^n(x)) \geq n-1$.



- Por otra parte se puede probar que $t(Th_2^n(x)) = O(1)$.

$$\underbrace{x_1 \ x_2 \ x_3}_{z = Th_2^3(x_1, x_2, x_3)}, \quad \underbrace{x_4 \ x_5 \ x_6}_{Th_2^3(x_4, x_5, x_6)} \quad \dots \quad x_n$$

can be done
with 6 gates

||

$\Downarrow$
puede computarse
en tiempo 6.

Primero, computamos $z = Th_2^3(x_1, x_2, x_3)$, después $z = Th_2^3(x_4, x_5, x_6)$ y así sucesivamente. Cada nodo que resetea z es un nodo de peso 6. De esta forma, las computaciones sobre $\underbrace{4 \cdot 2^{n-3}}_{\substack{\text{costo} \\ \text{sobre} \\ \{0,1\}^3}} = 2^{n-1}$ inputs se definen en 6 pasos,

(9)

las computaciones sobre $4^2 \cdot 2^{n-6} = 2^{n-2}$ ~~los~~ inputs restantes se detienen en
 $6:2 = 12$ pasos y en general, computaciones sobre $4^t 2^{n-3t} = 2^{n-t}$
inputs se detienen después de $6t$ pasos.

$\Rightarrow$ el tiempo de computación promedio es a lo más

$$\sum_{t=1}^{n/3} 6t 2^{-t} = O(1).$$